EXPERIMENT NO: 5

TITLE: PLC Programming for Industrial Automation Basic logic operations and motor control using PLC

AIM: To develop and simulate ladder logic programs to perform basic logic operations (AND,OR,NOT,NAND,NOR,XOR, XNOR) and motor control using a Programmable Logic Controller (PLC) in an industrial automation setup, and to design and implement a PLC-based control system for a conveyor belt.

APPARATUS:

Sr.No.	Equipment	Specification	Quantity
1	Programmable Logic Controller (PLC)	ABB AC 500 (PM554-ETH)	1
2	PLC Trainer Kit	ON/OFF Buttons (NO type)	2
		LED Lamp	1
3	PLC Simulator	Online PLC Simulator	Online Software

THEORY: Ladder logic (also known as ladder diagram or LD) is a programming language used to program a PLC (Programmable Logic Controller). It is a graphical PLC programming language which expresses logic operations with symbolic notation. Ladder logic is made out of rungs of logic, forming what looks like a ladder – hence the name 'Ladder Logic'.

Ladder logic is mainly for bit logic operations, although it is possible to scale a PLC analog input. Even simple bit logic operations can be beneficial in more advanced PLC programs and SCADA system programming.

Ladder logic is a graphical programming language which means that instead of text, the programming is done by combining different graphic elements. These graphic elements are called symbols. One of the smart things about the ladder logic symbols is that they are made to look like electrical symbols. Ladder logic was originally created for technicians, electricians, and people with an electrical background.

1. Easier to read

First of all, it makes ladder logic easier to read because it is natural for the eye to go from the left to right and then down to the next line. Just like when you are reading. Of course, this applies only to people living in countries where the reading is done from left to right.

2. Drawn on computer

When you draw ladder logic on a computer you will make one line at a time. As you draw more and more lines (in ladder logic called rungs) they will stack on top of each other, making up what looks like a ladder. The best way to look at a large ladder diagram with many lines is to scroll vertically along with the screen.

3. Order of execution

The last reason for drawing ladder logic vertically is to set the order of execution. The order of execution is how the PLC will run your ladder logic. To be more precise in what order your ladder logic instructions will be executed by the PLC. A PLC will always start at the top of your ladder logic and then execute its way down.

PROCEDURE:

1) First select **ABB Automation Builder** and double click on it.
2) Double click on "New project" and select AC500 Project then select the "Project name" and select the location path to save project and then click "OK".
3) Select "AC500 PM554-ETH" and Click "Add PLC".
4) Double Click on "Application"
5) Click on "Project" and select and click on "Rebuild All".
6) Right Click on "PLC_PRG" and Select "Convert Object" and Select Target Language "LD" and Click OK and Double Click on "PLC_PRG".
7) For XIC Command, click on **"Contact"** and for XIO Command, click on **"Contact(negated)"** Button on component toolbar
8) For Output Coil, Click on **"Coil"** Button on component toolbar.

LADDER LOGIC PROGRAMMING WITH INSTRUCTIONS

They are basically logic instructions that make you able to create a piece of logic. That piece of logic is your ladder logic or PLC program.

1) Examine if Closed (XIC): The first instruction here is called Examine If Closed. The symbol for instruction looks like this:

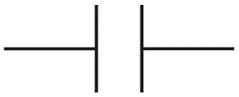


Fig. 1: Examine If Closed Instruction

This is a conditional instruction. It means that you can use it to check if something is true. For example, check if a bit is on.

Examine if closed is also known as normally open. It works basically the same way as a normally open contact in an electrical circuit. Of course, the normally open contact has no memory bit as a condition. The condition is whether the contact is activated or not. So the condition could be a finger pressing a button. The main point here is that each instruction has to be assigned an address in the PLC.

Working:

- When the PLC scan cycle starts, the PLC will check the states of all its inputs.
- It will then write in memory the Boolean value for these states (0 or 1).
- If an input is LOW, the bit will be set to 0.
- And if the input is HIGH the memory bit will be set to 1

LADDER DIAGRAM:

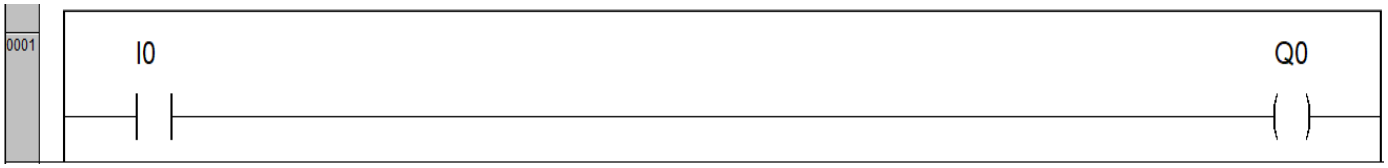


Fig. 2: Ladder Diagram for XIC Command

2) Examine if Open (XIO)



Fig. 3: Examine if Open Instruction

This instruction works the exact opposite way of the examine if closed instruction. The result of this instruction will be an inverted condition. It simply means

that, if the condition is "0" the result will be "1". Vice versa of course, so with condition "1" the result will be "0". To turn off the output coil we must somehow give it the condition "0".

Ladder Diagram:

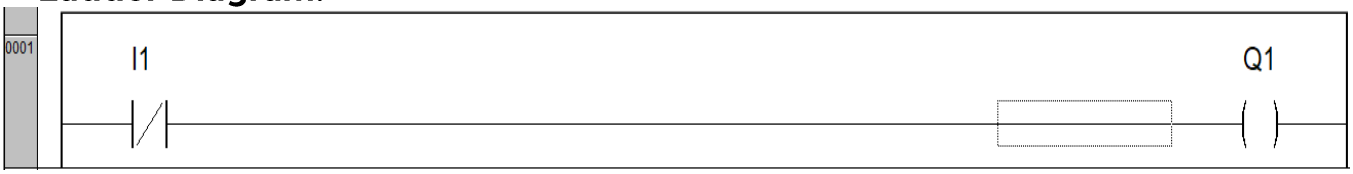


Fig. 4: Ladder Diagram using XIO

Working:

- When the PLC scan cycle starts, the PLC will check the states of all its inputs.
- It will then write in memory the Boolean value for these states (0 or 1).
- If an input is LOW, the bit will be set to 1.
- And if the input is HIGH the memory bit will be set to 0.

3) Output Coil



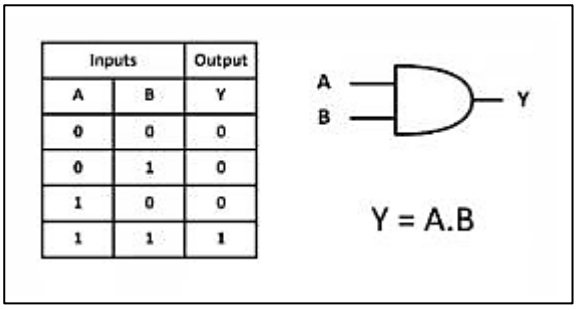
Fig. 5: Ladder Logic Output Coil

The instruction itself even has a place in the PLC memory. What the PLC will put there is the result of the instruction. An output coil is used to turn a bit on and off. The symbol is placed on the right side of the rung. The output coil uses the result of the previous instruction as a condition. This is called the RLO (Result of Logic Operation). The RLO is stored in a special place in PLC memory.

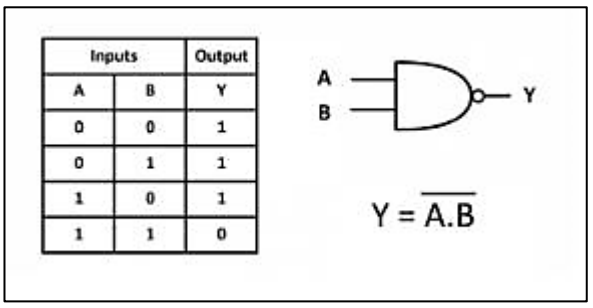
Logic Operations→

In industrial automation, Programmable Logic Controllers (PLCs) perform decision-making operations using logic gates similar to those in digital electronics. These logic operations form the foundation of all ladder logic programs used for process control, motor control, and automation tasks.

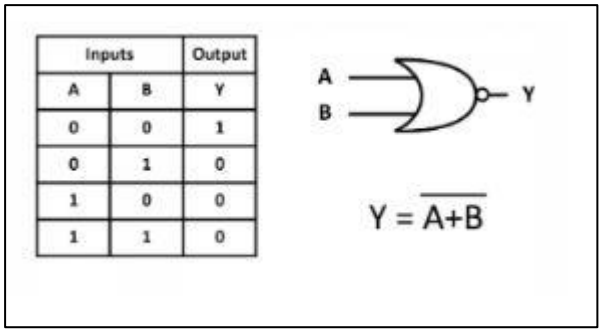
The logic operations are AND, OR, NOT, NAND, NOR, XOR. XNOR which can be implemented using contacts and coils in ladder logic diagrams.



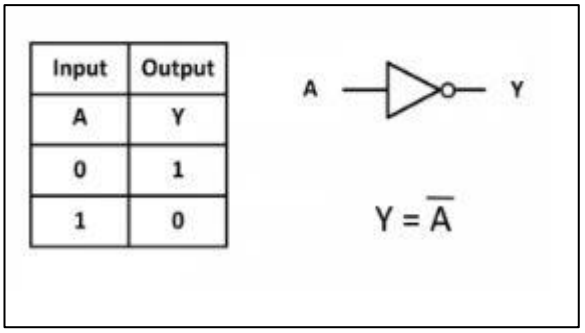
(a) AND Gate



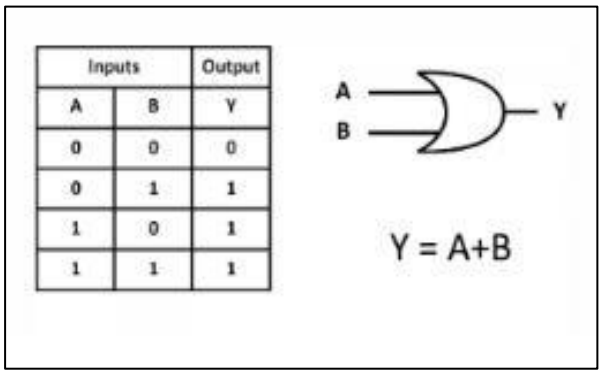
(b) NAND Gate



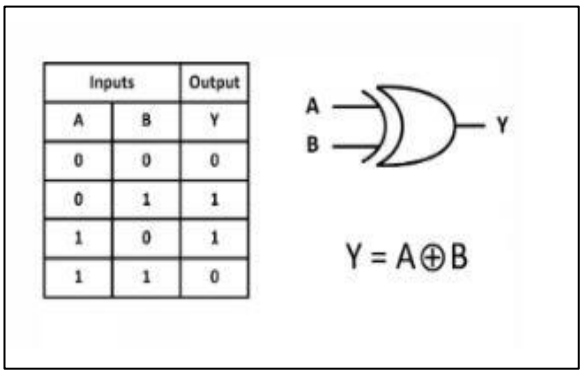
(c) NOR Gate



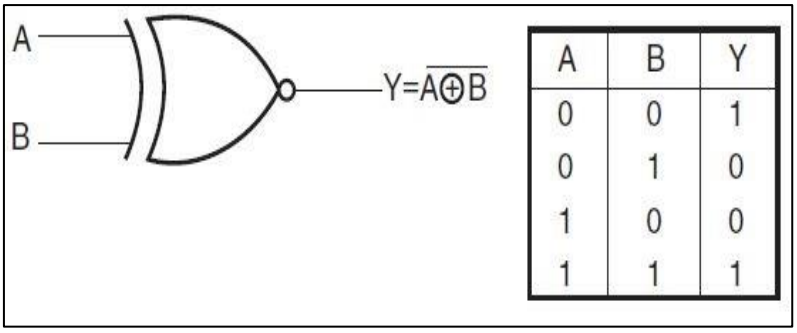
(d) NOT Gate



(e) OR Gate



(f) XOR Gate



(g) XNOR Gate

Motor Control Using PLC (Conveyer Belt)→

In this system, a PLC is used to automate the start, stop, and direction control of a conveyor belt motor. The PLC receives input signals from push buttons and sensors and controls the motor output accordingly. Ladder logic is designed to ensure safe and reliable operation, allowing automatic or manual control of the conveyor belt for efficient material handling.

Tag Names Summary (Left Panel)

Tag Name	Type	Description
X0	Bool	Input trigger (start pushbutton)
Z0, Z1, Z2	Bool	Outputs - likely motor or actuator steps
TimeOnZ0	Time	Timer for Z0 (3000 ms)
TimeOnZ1	Time	Timer for Z1 (5000 ms)
TimeOnZ2	Time	Timer for Z2 (7000 ms)
SystemOn	Bool	Main ON condition (Set/Reset logic)
2ClickTmr	Timer	Timer used between clicks
Counter	Counter	Counts clicks
Cycle	Timer	Main cycle timer (16000 ms)
2Clicks	Bool	Becomes true when 2 clicks done

Rung-by-Rung Explanation

Rung 1: [Start Condition]

```
scss
CopyEdit
X0
----[/]-----------------------------(SystemOn) Set
```

- **Action**: When input X0 is pressed (button pushed), SystemOn bit is **SET**.

Rung 2: [Reset Condition]

```
scss
CopyEdit
X0
----[ ]-----------------------------(SystemOn) Reset
```

- **Possibly** resets on same input – might be a bug or toggling logic (needs clarification).
- It seems like a toggle-type logic for SystemOn.

Rung 3: [Check for 2 Clicks]

```
scss
CopyEdit
X0
----[ ]-----------------------------(Counter) Count Up
```

- Every time X0 is pressed, counter increments.
- **Counter preset is 2** (PV=2), so after 2 presses → Counter.CV = 2.

Rung 4: [Timer After 2 Clicks]

```
diff
CopyEdit
Counter.CV == 2
----[ ]-----------------------------(2ClickTmr) Start TON (5s)
```

- Once **2 clicks** are detected, this starts a **5-second delay timer**.

Rung 5: [System ON + Delay Done = Start Cycle]

```
scss
CopyEdit
SystemOn AND 2ClickTmr.Q
----[ ]----[ ]----------------------(Cycle) TON (16s)
```

- When system is ON **and** the 5-sec timer is done, it starts the main **Cycle** timer for 16 seconds.
- This is the main control timer.

Rung 6, 7, 8: [Z0, Z1, Z2 Output Timings]

These rungs create **step-based outputs** during the 16s cycle:

Rung 6:

```
pgsql
CopyEdit
Cycle.ET >= 0
----[ ]------------------------------(Z0)
```

- Turn on Z0 immediately.

Rung 7:

```
pgsql
CopyEdit
Cycle.ET >= TimeOnZ0 (3000ms)
----[ ]------------------------------(Z1)
```

- After 3 seconds, Z1 turns ON.

Rung 8:

```
pgsql
CopyEdit
Cycle.ET >= TimeOnZ1 (5000ms)
----[ ]------------------------------(Z2)
```

- After 5 seconds, Z2 turns ON.

Summary

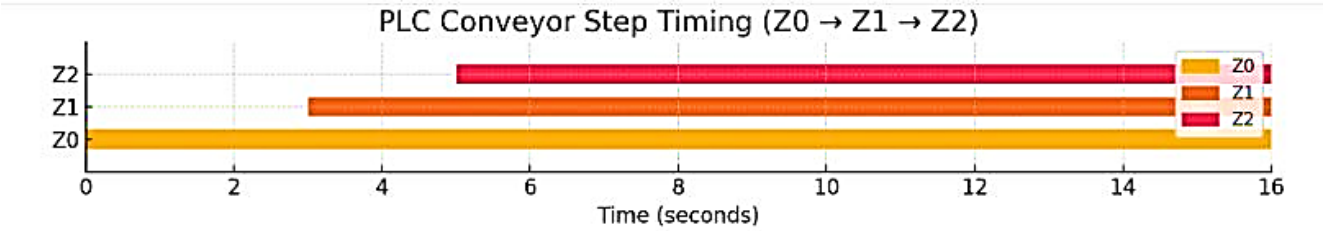
This ladder logic is an example of **stepwise timed motor control**, often used in **conveyor systems** or **automated stages**. The program likely represents a 3-step process:

1. **Press Button (X0) Twice** → system waits 5 seconds.
2. Then, a **16-second cycle starts**:
    - At 0s → **Z0 ON**
    - At 3s → **Z1 ON**
    - At 5s → **Z2 ON**
3. After 16s, cycle ends (not shown: reset logic).

Application

This program is suitable for:

- **Sequential motor control** (e.g., different conveyor zones)
- **Packaging systems**
- **Timed actuator stages**



Here's a **Gantt chart** that visually represents the **PLC conveyor step timing** for outputs `Z0`, `Z1`, and `Z2`:

Process Timeline Breakdown:

- **Z0** starts at **0 seconds** and runs for the entire cycle (until 16s).
- **Z1** starts at **3 seconds**, indicating a delay from Z0, and continues to the end.
- **Z2** starts at **5 seconds**, indicating another step forward, and runs to the end.

This type of sequencing is ideal for:

- **Conveyor belts** activating sections one after another.
- **Packaging stations** where each task starts after a delay.
- **Motorized systems** requiring staggered activation.

When the user presses the start button (X0) two times, the system recognizes this as a signal to begin operation. After detecting two button presses, the system waits for 5 seconds (this is done using a timer), and then it starts a full operation cycle that lasts for 16 seconds. During this 16-second cycle, three outputs—Z0, Z1, and Z2—turn ON one after the other based on timing:
Z0 turns ON After 3 seconds when the cycle starts.
Z1 turns ON while Z0 stays ON. Then after 5 seconds, Z1 turns ON, After 7 seconds, Z2 turns ON so now all three outputs (Z0, Z1, and Z2) are ON together until the end of the 16-second cycle.
This kind of control is useful in situations like conveyor systems or automated machines, where different motors or devices need to be activated at different times, in a step-by-step sequence.

RESULT:

Put the screenshot of the result of the experiment performed.(Each result page should contain Name, Roll No & Batch Name)

CONCLUSION:

Should be handwritten